

Übungsblatt 5: Inter-Task-Kommunikation und Interrupts

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Musterlösung

Zielsetzung

In dieser Übung verbinden wir die Hardware-Welt mit der Software-Welt des RTOS. Sie implementieren das in der Automobil- und Industriebranche weit verbreitete Architekturmuster des **Deferred Interrupt Processing** (Top-Half / Bottom-Half). Ein physischer Tasterdruck löst einen Hardware-Interrupt aus, welcher extrem kurz gehalten wird und über Semaphor bzw. Queue eine FreeRTOS-Task aufweckt. Diese Task übernimmt dann die zeitaufwändige Bearbeitung — inklusive einer kurzen Queue-Übung zur Inter-Task-Kommunikation.

Aufgabe 1: Hardware- und OS-Konfiguration im CubeMX

Zuerst richten wir den Hardware-Interrupt (EXTI), die Task und den Signalisierungs-Mechanismus ein.

1. Starten Sie ein Projekt für das Board **STM32H755ZI-Q**. Aktivieren Sie FreeRTOS für den Cortex-M4 (Interface **CMSIS_V2**). Stellen Sie unter *System Core* → *SYS* die HAL-Timebase auf **TIM6** um (SysTick bleibt FreeRTOS vorbehalten).
2. Konfigurieren Sie den blauen User-Button (Pin PC13) als **GPIO_EXTI13**.
3. Gehen Sie im CubeMX in die NVIC-Einstellungen und setzen Sie zwingend den Haken bei **EXTI line[15:10] interrupts** (globaler Interrupt aktivieren).
4. Legen Sie unter FreeRTOS im Reiter *Timers and Semaphores* einen **binären Semaphor** mit dem Namen **ButtonSem** an.
5. Legen Sie unter *Tasks and Queues* eine Task namens **ButtonTask** an und geben Sie ihr eine hohe Priorität (**osPriorityHigh**).
6. Generieren Sie den Code.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 1

Die Hardware-Konfiguration ist entscheidend. Ohne den Haken im NVIC wird das EXTI-Peripheriemodul das Interrupt-Signal niemals an den Cortex-M4-Kern (die CPU) weiterleiten. Der **binäre Semaphore** dient als reines Event-Signal ohne Ownership — er eignet sich damit ideal für die Signalisierung von der ISR (Top-Half) an die Task (Bottom-Half). Die **ButtonTask** erhält eine hohe Priorität, da Taster-Eingaben meist zeitkritisch auf die Benutzeroberfläche oder Steuerung durchschlagen sollen.

Aufgabe 2: Die Top-Half (Die Hardware-ISR)

Die ISR (Top-Half) reagiert auf den Hardware-Interrupt. Ihre einzige Aufgabe ist es, das Signal ins RTOS zu werfen und sich sofort zu beenden.

1. Suchen Sie in der `main.c` nach dem Benutzer-Block `USER CODE BEGIN 4`.
2. Überschreiben Sie dort die schwache (*weak*) HAL-Funktion `HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)`.
3. Prüfen Sie mit einer `if`-Bedingung, ob der übergebene Pin wirklich der Taster-Pin (`USER_Btn_Pin`) ist.
4. Rufen Sie die CMSIS-Funktion zur Freigabe des Semaphors auf. Was ist die Besonderheit des CMSIS-Wrappers im Vergleich zur nativen FreeRTOS-API in einer ISR?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 2

Code-Implementierung:

```
/* USER CODE BEGIN 4 */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == USER_Btn_Pin) {
        /* Wecke die Bottom-Half-Task auf */
        osSemaphoreRelease(ButtonSemHandle);
    }
}
/* USER CODE END 4 */
```

Besonderheit von CMSIS in der ISR:

In der nativen FreeRTOS-API müssten wir hier zwingend `xSemaphoreGiveFromISR()` aufrufen und anschließend das manuelle Yielding über `portYIELD_FROM_ISR` anfordern. Der CMSIS-Wrapper `osSemaphoreRelease()` ist kontextsensitiv: Er liest das IPSR-Hardware-Register (Interrupt Program Status Register) der ARM-CPU. Befinden wir uns im Handler Mode (`IPSR ≠ 0`), leitet CMSIS den Aufruf intern automatisch auf die sichere `...FromISR`-Variante um und fordert bei Bedarf den **PendSV**-Interrupt an, damit nach Beendigung der ISR unmittelbar in die geweckte höherpriori Task gewechselt wird. Im Thread Mode (`IPSR = 0`) bleibt der normale Task-Pfad.

Aufgabe 3: Die Bottom-Half (Die RTOS-Task)

Die ButtonTask (Bottom-Half) übernimmt nun die eigentliche, zeitaufwändige Arbeit, ohne den Hardware-Interrupt-Kontext zu blockieren.

1. Suchen Sie die generierte Funktion `StartButtonTask` in Ihrer `main.c`.
2. Sorgen Sie dafür, dass die Task am Anfang der `for(;;)`-Schleife 0% CPU-Zeit verbraucht, bis der Semaphor von der ISR freigegeben wird (`osSemaphoreAcquire(..., osWaitForever)`).
3. Wenn die Task aufwacht, schalten Sie zur Visualisierung eine LED (z. B. an Pin **PE1**) um und geben Sie via `printf` einen Text aus.
4. (Optional) Fügen Sie ein blockierendes `HAL_Delay(100)` ein, um eine schwere, 100 ms dauernde Berechnung zu simulieren. Warum führt dieses Delay hier im RTOS-Kontext nicht zum Systemabsturz, wohl aber, wenn es direkt in der ISR stehen würde?

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 3

Code-Implementierung:

```
/* USER CODE BEGIN Header_StartButtonTask */
void StartButtonTask(void *argument)
{
    for (;;) {
        /* 1. Schlafen (0 % CPU), bis die ISR signalisiert */
        osSemaphoreAcquire(ButtonSemHandle, osWaitForever);

        /* 2. WACH - hier die aufwändige Arbeit (Task-Kontext) */
        HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_1);
        printf("BUTTON PRESSED! Führe komplexe Berechnungen aus...\n");

        /* Demo-Last (Busy-Wait): Task bleibt Running - nur zur Demonstration!
         * In Produktion: osDelay(100); // Ticks; bei configTICK_RATE_HZ = 1000 → 100 ms
         */
        HAL_Delay(100);
    }
}
/* USER CODE END Header_StartButtonTask */
```

Erklärung zum Delay:

Stünde `HAL_Delay(100)` direkt in der ISR (Top-Half), wäre die CPU für 100 ms auf Hardware-Ebene gebunden. Wichtige RTOS-Interrupts wie der **SysTick** (Herzschlag des Kernels) könnten — je nach NVIC-Priorisierung — verzögert oder faktisch blockiert werden. Das Betriebssystem „friert“ ein: Deadlines verschieben sich, der Scheduler verliert seinen Takt.

Durch das **Deferred Interrupt Processing** lagern wir die Last in die RTOS-Task (Bottom-Half) aus. Dort läuft der SysTick normal weiter. `HAL_Delay()` ist zwar eine Busy-Wait-Schleife und hält die Task im Zustand *Running* (CPU-Last $\approx 100\%$ — in Produktion ungeeignet; dort `osDelay()`, damit die Task in *Blocked* geht). Dennoch könnte eine Realtime-Task mit

noch höherer RTOS-Priorität die `ButtonTask` jederzeit präemptieren. Das System bleibt unter Scheduler-Kontrolle und damit stabil.

Aufgabe 4: Mini-Queue (ITC aus der Vorlesung)

Ergänzen Sie das Semaphor-Muster um eine kurze Queue — so üben Sie die Inter-Task-/ISR-Kommunikation mit Payload.

1. Legen Sie in CubeMX eine Queue `ButtonQueue` an (Message Size: `uint32_t`, Länge ≥ 4).
2. In der ISR: schreiben Sie einen Zeitstempel (`HAL_GetTick()`) per `osMessageQueuePut(..., 0)` (Timeout 0!) in die Queue **und** geben Sie weiterhin den Semaphor frei.
3. In der Task: lesen Sie den Wert per `osMessageQueueGet` und geben Sie ihn mit `printf` aus.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 4

```
/* ISR (Top-Half) - zusätzlich zur Semaphor-Signalisierung */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_13) {
        uint32_t ts = HAL_GetTick();
        osMessageQueuePut(ButtonQueueHandle, &ts, 0, 0); /* Timeout 0 in ISR! */
        osSemaphoreRelease(ButtonSemHandle);
    }
}

/* Task - nach Acquire den Payload holen */
uint32_t ts = 0;
osMessageQueueGet(ButtonQueueHandle, &ts, NULL, 0);
printf("Button @ tick %lu\n", (unsigned long)ts);
```

Die Queue transportiert Daten (Copy-by-Value), der Semaphor bleibt das reine Wake-up-Signal. In Produktion reicht oft die Queue allein (Get mit `osWaitForever`); hier bleiben beide Muster sichtbar.